Software Design Document (SDD)

Module: GPIO_stm32 - GPIO Driver

Item	Description
Version	1.0
Authors	Steven McClellan, Vania Leal, Leonardo Garcia, Melanie Picen
Source file	GPIO_stm32.c
Main purpose	GPIO port clock enable, pin mode configuration, digital output control, input reading, alternate function configuration, open-drain selection, and pull-up / pull-down configuration for STM32 GPIO ports.
Main public functions	gpio_init(), gpio_initPort(), gpio_setPinMode(), gpio_setPin(), gpio_clearPin(), gpio_togglePin(), gpio_readPin(), gpio_setAlternateFunction(), gpio_setOpenDrain(), gpio_setPullUp(), gpio_setPullDown()

1. Introduction

1.1 Purpose

This document describes the design and architecture of the GPIO_stm32 driver module. The module provides a low-level API for configuring STM32 GPIO ports and pins without requiring application code to manipulate GPIO registers directly. It covers port clock activation, pin mode selection, digital output write operations, digital input reading, alternate function assignment, open-drain output configuration, and internal pull-up / pull-down configuration.

1.2 Scope

The GPIO_stm32 driver is a low-level embedded software component. It directly accesses RCC and GPIO registers through definitions provided by GPIO_stm32.h. The implementation supports STM32 GPIO ports A, B, C, D, E, and H, while rejecting reserved or unavailable port enum values 5 and 6. The driver validates ports, pins, modes, clock state, and output pointers before performing register operations.

Only GPIO_stm32.c was provided for this document. Therefore, enum numeric values and register macros are documented from how they are used in the source file. The companion GPIO_stm32.h header is assumed to declare the public types, status codes, port definitions, and register structures referenced by the source.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
GPIO	General Purpose Input/Output. Digital pins that can be configured as input, output, alternate function, or analog.
RCC	Reset and Clock Control. Peripheral used to enable GPIO port clocks through RCC_AHB1ENR.
AHB1	Advanced High-performance Bus 1. STM32 bus where GPIO peripherals are clocked.
MODER	GPIO mode register. Uses two bits per pin to select input, output, alternate function, or analog mode.
BSRR	GPIO bit set/reset register. Lower 16 bits set outputs high; upper 16 bits reset outputs low.
ODR	Output data register. Holds the current output states and is used by gpio_togglePin().
IDR	Input data register. Holds sampled input states and is used

	by gpio_readPin().
AFR	Alternate function registers. AFR[0] configures pins 0-7; AFR[1] configures pins 8-15.
OTYPER	Output type register. Selects push-pull or open-drain output type.
PUPDR	Pull-up / pull-down register. Selects no pull, pull-up, or pull-down for each pin.
SDD	Software Design Document.

1.4 Requirements Traceability

Req. ID	Function	Requirement Statement
FR-1	gpio_init()	The system shall provide a GPIO initialization entry point and return GPIO_OK.
FR-2	gpio_initPort()	The system shall validate a GPIO port and enable its AHB1 clock through RCC_AHB1ENR.
FR-3	gpio_setPinMode()	The system shall configure the selected pin mode by writing the correct two-bit field in MODER.
FR-4	gpio_setPin()	The system shall drive an output pin high by writing the lower half of BSRR.
FR-5	gpio_clearPin()	The system shall drive an output pin low by writing the upper half of BSRR.
FR-6	gpio_togglePin()	The system shall invert an output pin state using ODR.
FR-7	gpio_readPin()	The system shall read an input pin from IDR and return the value through a caller-provided pointer.
FR-8	gpio_setAlternateFunction()	The system shall program the correct AFR register and field for a pin already configured as alternate function.
FR-9	gpio_setOpenDrain()	The system shall enable open-drain output type for pins configured as output or alternate function.
FR-10	gpio_setPullUp()	The system shall configure the selected pin with internal pull-up resistance through PUPDR.
FR-11	gpio_setPullDown()	The system shall configure the selected pin with internal pull-down resistance through PUPDR.

Non-functional requirements: no dynamic memory allocation; direct register access only through the configured register definitions; defensive validation of port, pin, mode, alternate function, and pointer parameters; clear error code returns for invalid or unsafe operations.

2. System Architecture

2.1 High-Level Design

The GPIO driver follows a layered design. Application code calls public GPIO API functions. The GPIO driver validates arguments, checks whether the selected port clock has been enabled, resolves the selected port into a GPIO_RegDef_t pointer, and updates the appropriate hardware register.

Layer	Files / Components	Description
Application Layer	main.c or higher-level modules	Calls gpio_initPort(), gpio_setPinMode(), gpio_setPin(), gpio_readPin(), and other public GPIO APIs.

GPIO Driver Layer	GPIO_stm32.c / GPIO_stm32.h	Performs parameter validation, port resolution, and register-level GPIO operations.
Register Layer	RCC_AHB1ENR and GPIOx registers	Stores peripheral clock state and pin configuration/output/input values.
Hardware Layer	STM32 GPIO ports and pins	Physical MCU pins connected to LEDs, buttons, I2C/UART alternate functions, or other external circuits.

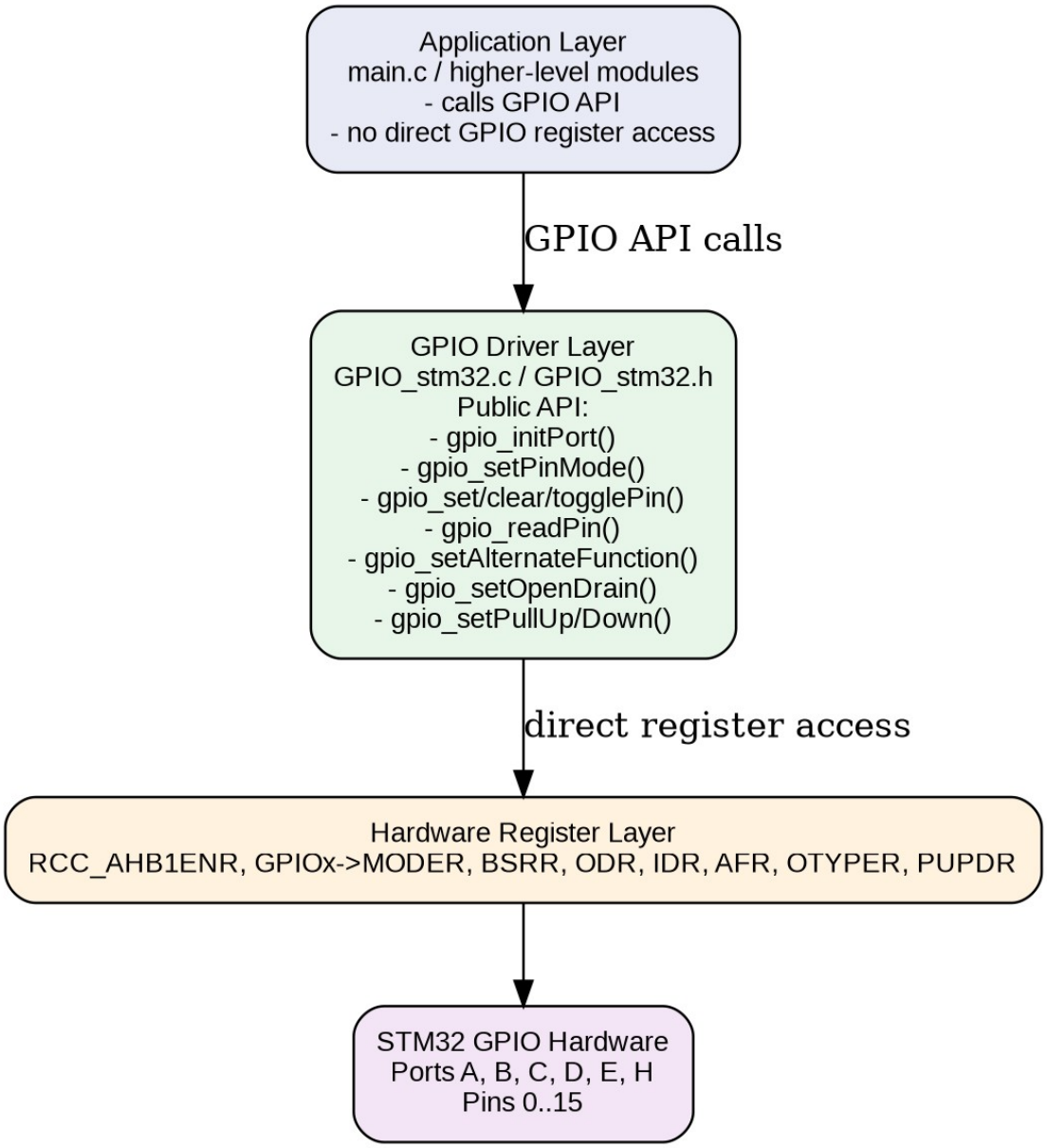

Figure 1. Class Diagram - GPIO_stm32 driver layered structure.

2.2 Module Interfaces

Interface	Description
Public API	Functions declared in GPIO_stm32.h and implemented in GPIO_stm32.c.
External dependencies	stdint.h, stddef.h, GPIO_stm32.h, RCC_AHB1ENR, GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOH, GPIO_RegDef_t.
Register dependencies	RCC_AHB1ENR, GPIOx->MODER, GPIOx->BSRR, GPIOx->ODR, GPIOx->IDR, GPIOx->AFR[], GPIOx->OTYPER,

	GPIOx->PUPDR.
Public data types	gpio_status_t, gpio_port_t, gpio_pin_t, gpio_mode_t, gpio_altfunction_t.

3. Detailed Design

3.1 Module Behavior

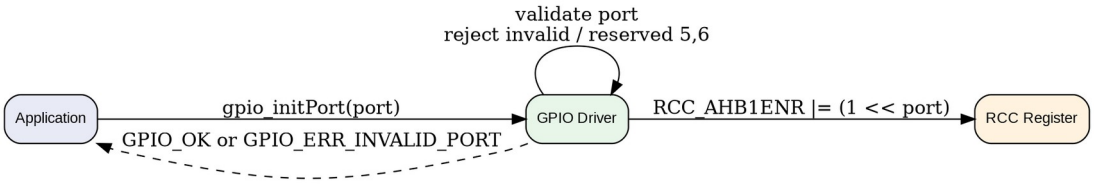

Figure 2. Sequence Diagram - gpio_initPort.

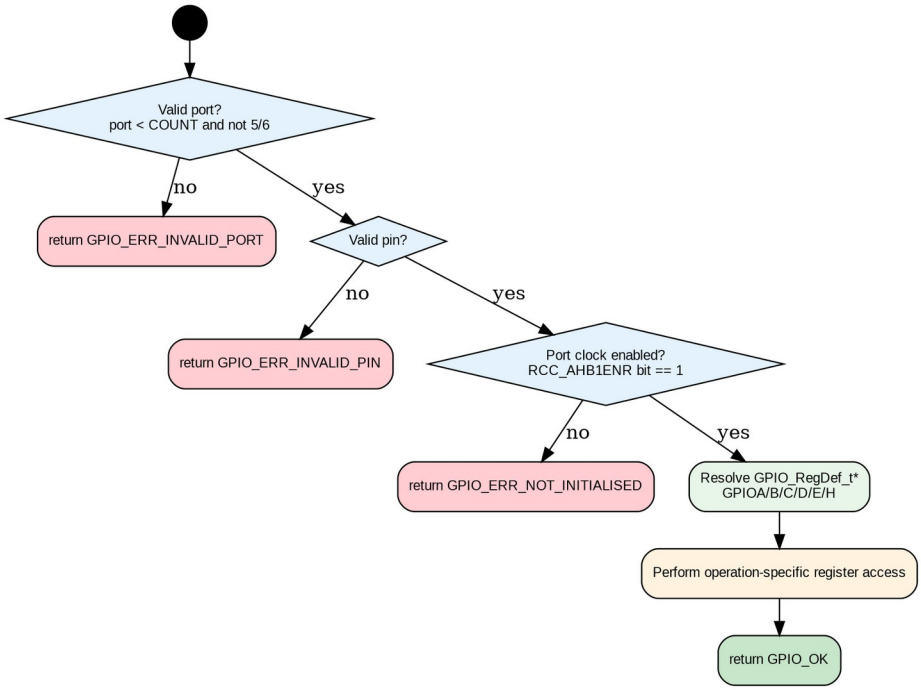

Figure 3. Activity Diagram - Common validation and port resolution pattern.

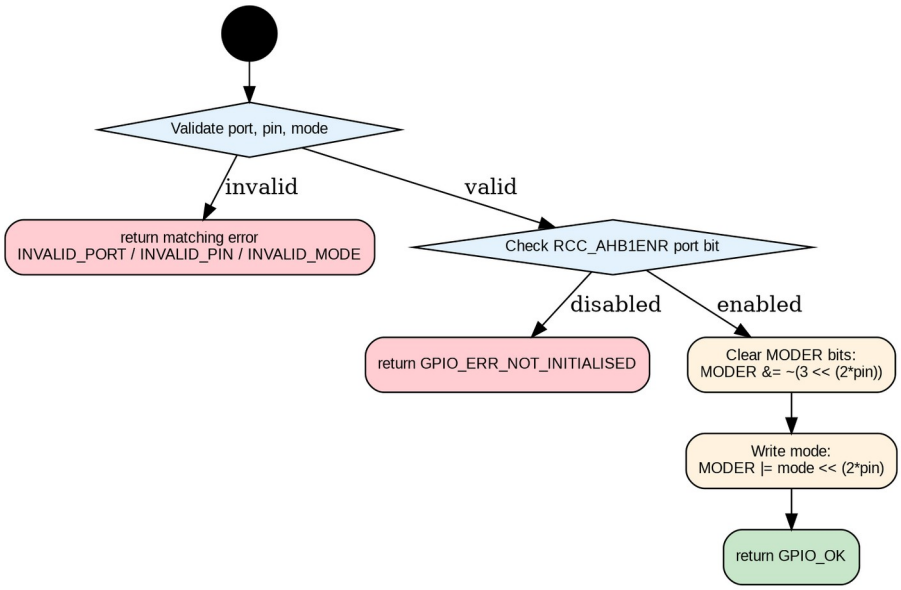

Figure 4. Activity Diagram - gpio_setPinMode.

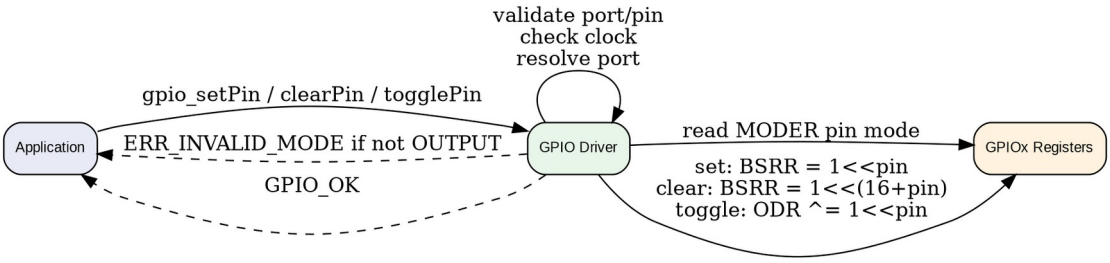

Figure 5. Sequence Diagram - Output pin operations.

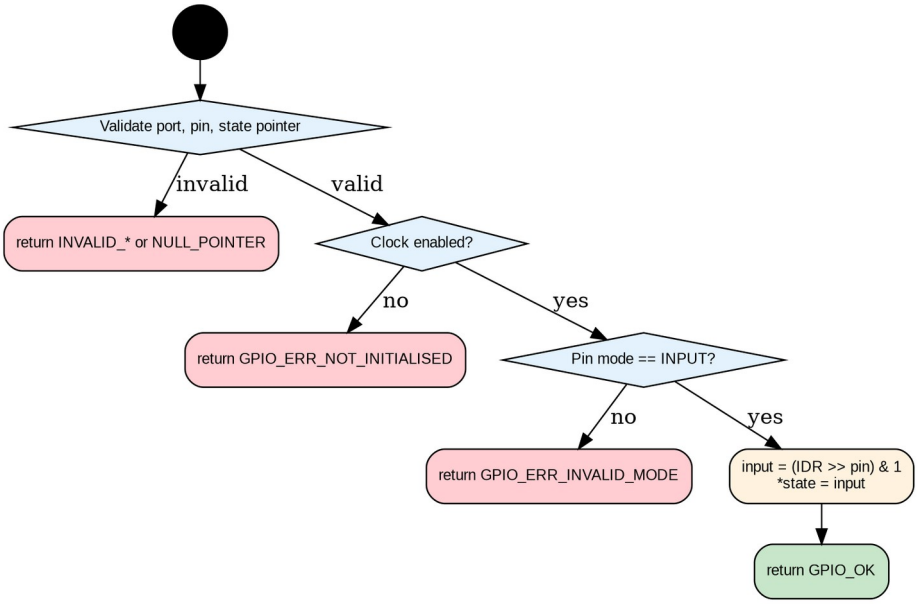

Figure 6. Activity Diagram - gpio_readPin.

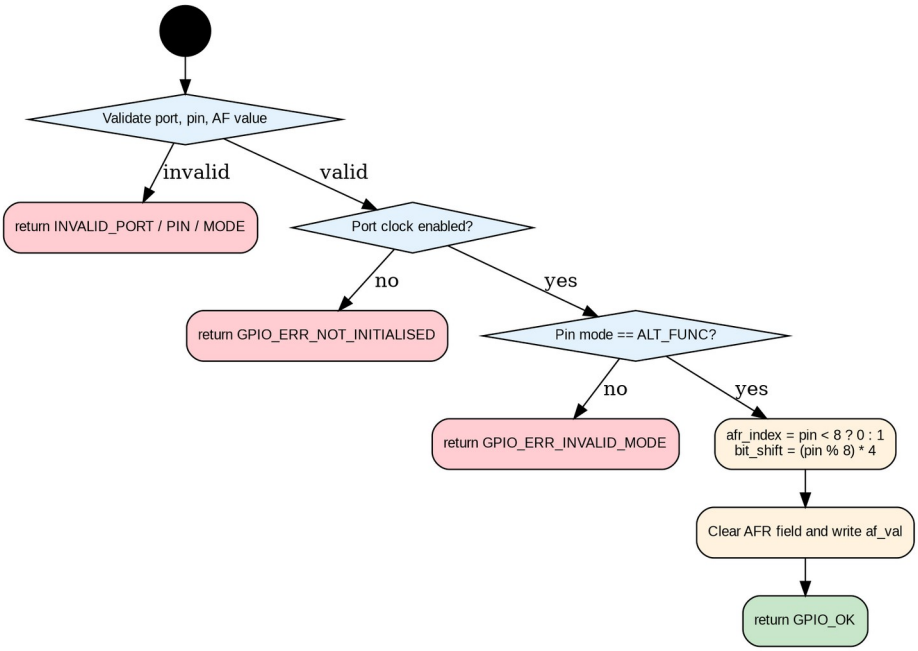

Figure 7. Activity Diagram - gpio_setAlternateFunction.

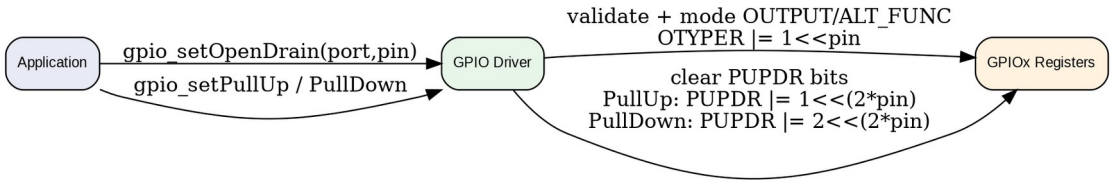

Figure 8. Sequence Diagram - Open-drain and pull resistor configuration.

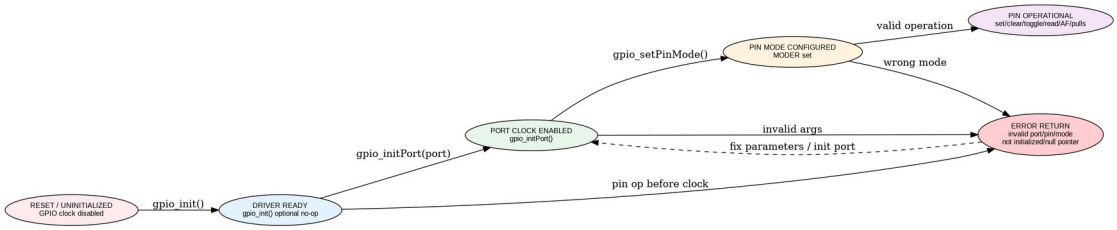

Figure 9. State Diagram - GPIO driver lifecycle.

3.2 GPIO Driver Design

The driver uses direct register manipulation after validating that the selected port and pin are legal and that the target port clock is enabled. Port validation rejects enum values outside GPIO_PORT_COUNT and also rejects port values 5 and 6, leaving ports A, B, C, D, E, and H as valid targets. After validation, a switch statement maps the logical port enum to the corresponding GPIO_RegDef_t pointer.

3.3 Port Initialization

gpio_init() is a no-operation initialization entry point and returns GPIO_OK. gpio_initPort() performs the actual port enable operation by setting the corresponding bit in RCC_AHB1ENR. All pin-level operations check this bit before accessing GPIO registers, returning GPIO_ERR_NOT_INITIALISED if the selected port clock has not been enabled.

3.4 Pin Mode Configuration

gpio_setPinMode() configures the MODER field of the selected pin. Each pin uses two bits in MODER, so the function clears the target field using ~(3U << (2U * pin)) and writes the requested mode using mode << (2U * pin). The function validates port, pin, mode, and clock enable state before modifying MODER.

3.5 Digital Output Operations

gpio_setPin(), gpio_clearPin(), and gpio_togglePin() require the target pin to already be configured as GPIO_MODE_OUTPUT. gpio_setPin() writes 1U << pin into BSRR to set the output high. gpio_clearPin() writes 1U << (16U + pin) into BSRR to reset the output low. gpio_togglePin() reads and modifies ODR using XOR. The BSRR-based set and clear operations are atomic at the register level, while ODR toggle is a read-modify-write operation.

3.6 Digital Input Reading

gpio_readPin() requires the caller to provide a non-NULL uint8_t pointer and requires the selected pin to be configured as GPIO_MODE_INPUT. The function reads the selected bit from IDR by shifting right by the pin number and masking with 1U. The resulting value is written through the caller-provided pointer.

3.7 Alternate Function Configuration

gpio_setAlternateFunction() validates the alternate function value and requires the pin mode to already be GPIO_MODE_ALT_FUNC. It selects AFR[0] for pins 0-7 or AFR[1] for pins 8-15. The selected four-bit field is cleared and then loaded with the requested alternate function value.

3.8 Electrical Configuration Helpers

gpio_setOpenDrain() sets the OTYPER bit for a pin, but only when the pin is configured as GPIO_MODE_OUTPUT or GPIO_MODE_ALT_FUNC. gpio_setPullUp() and gpio_setPullDown() configure the PUPDR two-bit field for the target pin after validating port, pin, and clock state. Pull-up writes 01 to the field, while pull-down writes 10.

4. Application Programming Interfaces

4.1 Data Types and Macros

Type / Macro	Category	Notes / Definition
gpio_status_t	Return status enum	Represents success or specific validation/configuration errors.
gpio_port_t	Port enum	Logical GPIO port identifier. Valid implementation targets are GPIO_PORT_A, B, C, D, E, and H.
gpio_pin_t	Pin enum / integer type	Pin index. The implementation validates pin < GPIO_PIN_COUNT.
gpio_mode_t	Mode enum	GPIO_MODE_INPUT, GPIO_MODE_OUTPUT, GPIO_MODE_ALT_FUNC, and other modes defined by the header.
gpio_altfunction_t	Alternate function enum	Alternate function selector written into AFR fields. The implementation validates af_val < GPIO_AF_COUNT.
GPIO_RegDef_t	Register structure	Represents the GPIO register block, including MODER, BSRR, ODR, IDR, AFR, OTYPER, and PUPDR.
RCC_AHB1ENR	Register macro / lvalue	AHB1 peripheral clock enable register used to enable and verify GPIO port clocks.

4.2 Error Codes

Code	Notes / Definition
GPIO_OK	Operation completed successfully.
GPIO_ERR_INVALID_PORT	The selected port is outside GPIO_PORT_COUNT or maps to an unsupported reserved value 5 or 6.
GPIO_ERR_INVALID_PIN	The selected pin is outside GPIO_PIN_COUNT.
GPIO_ERR_INVALID_MODE	The requested mode / alternate function is invalid, or the current pin mode is incompatible with the requested operation.
GPIO_ERR_NOT_INITIALISED	The selected GPIO port clock is not enabled in RCC_AHB1ENR.
GPIO_ERR_NULL_POINTER	A required output pointer is NULL, used by gpio_readPin().

4.3 Function Definitions

gpio_init

Field	Definition
Function Name	gpio_init
Syntax	gpio_status_t gpio_init(void)
Parameters (in)	None
Parameters (out)	None
Return value	gpio_status_t
Description	No-op module initialization entry point. Returns GPIO_OK.
Constraints	Does not enable any port clock or configure any pin.
Available via	GPIO_stm32.h

gpio_initPort

Field	Definition
Function Name	gpio_initPort
Syntax	gpio_status_t gpio_initPort(gpio_port_t port)
Parameters (in)	port: GPIO port to enable.
Parameters (out)	None
Return value	gpio_status_t
Description	Enables the AHB1 clock for the selected GPIO port.
Constraints	Rejects invalid/reserved ports. Assumes port enum value matches the RCC_AHB1ENR bit position.
Available via	GPIO_stm32.h

gpio_setPinMode

Field	Definition
Function Name	gpio_setPinMode
Syntax	gpio_status_t gpio_setPinMode(gpio_port_t port, gpio_pin_t pin, gpio_mode_t mode)
Parameters (in)	port, pin, mode.
Parameters (out)	None
Return value	gpio_status_t
Description	Configures the selected pin mode in MODER.
Constraints	Port clock must already be enabled.
Available via	GPIO_stm32.h

gpio_setPin

Field	Definition
Function Name	gpio_setPin
Syntax	gpio_status_t gpio_setPin(gpio_port_t port, gpio_pin_t pin)
Parameters (in)	port, pin.
Parameters (out)	None

Return value	gpio_status_t
Description	Sets an output pin high using BSRR.
Constraints	Pin must be configured as GPIO_MODE_OUTPUT.
Available via	GPIO_stm32.h

gpio_clearPin

Field	Definition
Function Name	gpio_clearPin
Syntax	gpio_status_t gpio_clearPin(gpio_port_t port, gpio_pin_t pin)
Parameters (in)	port, pin.
Parameters (out)	None
Return value	gpio_status_t
Description	Clears an output pin low using BSRR upper half.
Constraints	Pin must be configured as GPIO_MODE_OUTPUT.
Available via	GPIO_stm32.h

gpio_togglePin

Field	Definition
Function Name	gpio_togglePin
Syntax	gpio_status_t gpio_togglePin(gpio_port_t port, gpio_pin_t pin)
Parameters (in)	port, pin.
Parameters (out)	None
Return value	gpio_status_t
Description	Toggles an output pin using ODR XOR.
Constraints	Pin must be configured as GPIO_MODE_OUTPUT. Operation is read-modify-write.
Available via	GPIO_stm32.h

gpio_readPin

Field	Definition
Function Name	gpio_readPin
Syntax	gpio_status_t gpio_readPin(gpio_port_t port, gpio_pin_t pin, uint8_t *state)
Parameters (in)	port, pin.
Parameters (out)	state: receives pin state.
Return value	gpio_status_t
Description	Reads an input pin from IDR and writes 0 or 1 to *state.
Constraints	state must not be NULL. Pin must be configured as GPIO_MODE_INPUT.
Available via	GPIO_stm32.h

gpio_setAlternateFunction

Field	Definition
Function Name	gpio_setAlternateFunction
Syntax	gpio_status_t gpio_setAlternateFunction(gpio_port_t port, gpio_pin_t pin, gpio_altfunction_t af_val)
Parameters (in)	port, pin, af_val.
Parameters (out)	None
Return value	gpio_status_t
Description	Writes the requested alternate function value into AFR[0] or AFR[1].
Constraints	Pin must already be configured as GPIO_MODE_ALT_FUNC.
Available via	GPIO_stm32.h

gpio_setOpenDrain

Field	Definition
Function Name	gpio_setOpenDrain
Syntax	gpio_status_t gpio_setOpenDrain(gpio_port_t port, gpio_pin_t pin)
Parameters (in)	port, pin.
Parameters (out)	None
Return value	gpio_status_t
Description	Sets the OTYPER bit to enable open-drain output.
Constraints	Pin must be configured as GPIO_MODE_OUTPUT or GPIO_MODE_ALT_FUNC.
Available via	GPIO_stm32.h

gpio_setPullUp

Field	Definition
Function Name	gpio_setPullUp
Syntax	gpio_status_t gpio_setPullUp(gpio_port_t port, gpio_pin_t pin)
Parameters (in)	port, pin.
Parameters (out)	None
Return value	gpio_status_t
Description	Configures the selected pin PUPDR field as pull-up.
Constraints	Does not validate pin mode; only validates port, pin, and clock state.
Available via	GPIO_stm32.h

gpio_setPullDown

Field	Definition
Function Name	gpio_setPullDown
Syntax	gpio_status_t gpio_setPullDown(gpio_port_t port, gpio_pin_t pin)
Parameters (in)	port, pin.
Parameters (out)	None
Return value	gpio_status_t
Description	Configures the selected pin PUPDR field as pull-down.
Constraints	Does not validate pin mode; only validates port, pin, and clock state.
Available via	GPIO_stm32.h

4.4 Internal Function Definitions

The current GPIO_stm32.c implementation does not define static private helper functions. Validation and port-resolution logic are repeated inside each public API function.

5. Implementation Notes and Constraints

Topic	Note
Port clock dependency	All pin-level functions require gpio_initPort() to be called first for the selected port. Otherwise, they return GPIO_ERR_NOT_INITIALISED.
Reserved ports	Port enum values 5 and 6 are explicitly rejected, while port H is accepted through the switch statement.
Mode-before-operation rule	Output write/toggle functions require GPIO_MODE_OUTPUT. Read requires GPIO_MODE_INPUT. Alternate function assignment requires GPIO_MODE_ALT_FUNC.
Atomicity	Set and clear use BSRR and are atomic register writes. Toggle uses ODR read-modify-write and is not atomic in the

	same way.
Header dependency	The exact enum numeric values and register structure definitions must match the assumptions in GPIO_stm32.c.
Code duplication	Port validation, clock checking, and port pointer resolution appear in multiple functions and could be centralized.

6. Future Enhancements

- Add a private helper function to validate port/pin and return the GPIO_RegDef_t pointer to reduce duplicated code.
- Add a helper to check whether a port clock is enabled.
- Add gpio_setPushPull() to complement gpio_setOpenDrain().
- Add gpio_clearPull() to explicitly remove pull-up/pull-down configuration.
- Add optional speed configuration through OSPEEDR if required by high-speed peripherals.
- Add unit tests or register-level mocks for validation paths and register writes.
- Document the exact enum values in GPIO_stm32.h to make the RCC_AHB1ENR bit mapping explicit.